

Day 7 — 实战项目：URL 短链服务（完整版）

一、项目规划

1. 是什么

今天不是学新知识，而是用之前 6 天学到的所有东西，从零搭建一个可以部署的 Web 服务。

我们做一个 **URL Shortener**（短链服务）——把长网址变成短码。

功能清单：

- `POST /shorten` — 提交长网址，返回短码
- `GET /{short_code}` — 访问短码，301 重定向到原网址
- `GET /stats/{code}` — 查看短码的访问统计
- CLI 管理工具 — 初始化数据库、列出短链、删除短链

2. 技术栈速览

技术	之前哪天学的	这里用来干嘛
FastAPI	Day 6	API 框架
Pydantic	Day 4	数据校验
SQLAlchemy	Day 6	数据库 ORM
Click	Day 6	CLI 管理工具
pytest	Day 6	测试

技术	之前哪天学的	这里用来干嘛
logging	Day 3	日志
secrets	Day 1	安全随机短码

3. 项目结构

```
url-shortener/  
├── src/  
│   ├── shortener/  
│   │   ├── __init__.py  
│   │   ├── __main__.py      # python -m shortener  
│   │   ├── app.py           # FastAPI 应用  
│   │   ├── models.py        # SQLAlchemy 模型  
│   │   ├── schemas.py       # Pydantic schema  
│   │   ├── database.py      # 数据库连接  
│   │   ├── cli.py           # Click 管理命令  
│   │   └── utils.py          # 工具函数  
├── tests/  
│   ├── __init__.py  
│   ├── test_api.py  
│   └── conftest.py  
├── pyproject.toml  
└── .gitignore
```

先搭好骨架再填充代码。这就是 Day 6 讲的 src 布局。

二、数据库层 — SQLAlchemy

```
# src/shortener/models.py  
from sqlalchemy import Column, Integer, String, DateTime, BigInteger, func  
from sqlalchemy.orm import declarative_base
```

```
Base = declarative_base()

class URL(Base):
    __tablename__ = "urls"

    id = Column(Integer, primary_key=True)
    original_url = Column(String(2048), nullable=False)
    short_code = Column(String(10), unique=True, index=True,
nullable=False)
    created_at = Column(DateTime, server_default=func.now())
    visits = Column(BigInteger, default=0)
```

```
# src/shortener/database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
import os

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///shortener.db")
engine = create_engine(DATABASE_URL, echo=False)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    """FastAPI 依赖注入用的函数。每次请求获取一个新 session。"""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

设计决策解析： - `short_code` 设了 `unique=True + index=True`：查找短码是核心操作，必须快 - `BigInteger` 存 `visits`：日活高的链接访问量可能很大 - `server_default=func.now()`：数据库自动填时间，应用层不用管 - 用 `DATABASE_URL` 环境变量：随时切换 SQLite → PostgreSQL，代码不用改

三、数据校验层 — Pydantic

```
# src/shortener/schemas.py
from pydantic import BaseModel, HttpUrl
from datetime import datetime

class ShortenRequest(BaseModel):
    """创建短链的请求体"""
    url: HttpUrl # Pydantic 自动校验 URL 格式

class ShortenResponse(BaseModel):
    """创建短链的响应体"""
    short_code: str
    short_url: str
    original_url: str

class URLStats(BaseModel):
    """短链统计的响应体"""
    short_code: str
    original_url: str
    visits: int
    created_at: datetime
```

为什么用三个 **Schema** 而不是一个： - 请求和响应需要的字段不同 - `HttpUrl` 类型自动校验：传入的不是合法 URL 直接 422，不用手写正则 - 各管各的，修改一个不影响其他

四、工具函数层

```
# src/shortener/utils.py
import secrets
import string

def generate_short_code(length: int = 6) -> str:
    """生成安全随机的短码"""
    alphabet = string.ascii_lowercase + string.digits
```

```

    return "".join(secrets.choice(alphabet) for _ in range(length))

def generate_unique_code(db_session, length=6):
    """生成不重复的短码"""
    from .models import URL

    while True:
        code = generate_short_code(length)
        exists = db_session.query(URL).filter(
            URL.short_code == code
        ).first()
        if not exists:
            return code

```

设计决策解析：- `secrets.choice` 替代 `random.choice`：密码学安全的随机数，不能被人猜出下一个短码 - `string.ascii_lowercase + digits`（36 个字符）：6 位有 $36^6 \approx 21$ 亿个组合 - 循环直到找到不重复的码：碰撞概率极低，实际基本一次命中

五、应用入口 — FastAPI

```

# src/shortener/app.py
import logging
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.responses import RedirectResponse
from sqlalchemy.orm import Session

from .models import Base, URL
from .schemas import ShortenRequest, ShortenResponse, URLStats
from .database import engine, get_db
from .utils import generate_unique_code

logger = logging.getLogger(__name__)

# 启动时自动建表
Base.metadata.create_all(bind=engine)

app = FastAPI(title="URL Shortener")

```

```

@app.post("/shorten", response_model=ShortenResponse, status_code=201)
def shorten_url(
    request: ShortenRequest,
    db: Session = Depends(get_db),
):
    """提交长网址，返回短码"""
    short_code = generate_unique_code(db)
    db_url = URL(
        original_url=str(request.url),
        short_code=short_code,
    )
    db.add(db_url)
    db.commit()
    db.refresh(db_url)
    logger.info(f"新短链: {short_code} → {request.url}")
    return ShortenResponse(
        short_code=short_code,
        short_url=f"http://short.url/{short_code}",
        original_url=str(request.url),
    )


@app.get("/{short_code}")
def redirect_to_url(
    short_code: str,
    db: Session = Depends(get_db),
):
    """访问短链 → 301 重定向"""
    url_entry = db.query(URL).filter(URL.short_code == short_code).first()
    if not url_entry:
        raise HTTPException(status_code=404, detail="短链不存在")
    url_entry.visits += 1
    db.commit()
    return RedirectResponse(url=url_entry.original_url, status_code=301)


@app.get("/stats/{short_code}", response_model=URLStats)
def get_stats(

```

```

    short_code: str,
    db: Session = Depends(get_db),
):
    """查看短链统计"""
    url_entry = db.query(URL).filter(URL.short_code == short_code).first()
    if not url_entry:
        raise HTTPException(status_code=404, detail="短链不存在")
    return URLStats(
        short_code=url_entry.short_code,
        original_url=url_entry.original_url,
        visits=url_entry.visits,
        created_at=url_entry.created_at,
    )

```

逐行解析：

代码	作用	为什么这样写
<code>Base.metadata.create_all(bind=engine)</code>	自动建表	开发时省一个步骤；生产要用迁移工具
<code>response_model=ShortenResponse</code>	FastAPI 自动序列化并校验响应	确保响应结构正确，自动生成 API 文档
<code>Depends(get_db)</code>	依赖注入，每次请求创建/关闭 session	确保 session 正确关闭；测试可替换
<code>status_code=201</code>	创建资源返回 201	RESTful 规范
<code>RedirectResponse(status_code=301)</code>	301 永久重定向	浏览器会缓存，下次直接跳转

六、CLI 管理工具 — Click

```
# src/shortener/cli.py
import click
from .database import SessionLocal, engine
from .models import Base, URL


@click.group()
def cli():
    """URL Shortener 管理工具"""
    pass


@click.command()
def init_db():
    """初始化数据库（建表）"""
    Base.metadata.create_all(bind=engine)
    click.echo("数据库已初始化 ✓")


@click.command()
def list_urls():
    """列出所有短链"""
    db = SessionLocal()
    urls = db.query(URL).all()
    if not urls:
        click.echo("暂无短链")
        db.close()
        return
    click.echo(f"{'短码':<12s} {'访问数':<8s} {'原始 URL'}")
    click.echo("-" * 60)
    for url in urls:
        click.echo(f"{url.short_code:<12s} {url.visits:<8d} {url.original_url}")
    db.close()


@click.command()
```



```

@click.argument("short_code")
def delete(short_code):
    """删除指定短链"""
    db = SessionLocal()
    url = db.query(URL).filter(URL.short_code == short_code).first()
    if not url:
        click.echo(f"未找到: {short_code}")
        db.close()
        return
    db.delete(url)
    db.commit()
    db.close()
    click.echo(f"已删除 {short_code} ✓")

if __name__ == "__main__":
    cli()

```

```

# 注册为命令行工具 (在 pyproject.toml 中配置)
shortener init-db          # 建表
shortener list-urls        # 列出所有
shortener delete abc123    # 删除

```

七、pyproject.toml + main.py

```

[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"

[project]
name = "url-shortener"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "fastapi>=0.110",
    "uvicorn>=0.27",
    "sqlalchemy>=2.0",

```

```
    "pydantic>=2.5",
    "click>=8.1",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "httpx>=0.27",
    "ruff>=0.5",
]

[project.scripts]
shortener = "shortener.cli:cli"

[tool.setuptools.packages.find]
where = ["src"]
```

```
# src/shortener/__main__.py
# 支持 python -m shortener 启动
from .cli import cli
cli()
```

八、测试

```
# tests/conftest.py
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from fastapi.testclient import TestClient
from shortener.models import Base
from shortener.database import get_db
from shortener.app import app

@pytest.fixture
def db_session():
    """每个测试用独立的内存数据库"""
```

```

    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(bind=engine)
    TestingSession = sessionmaker(bind=engine)
    session = TestingSession()
    yield session
    session.close()

@pytest.fixture
def client(db_session):
    """替换 FastAPI 的数据库依赖为测试数据库"""
    def override_get_db():
        yield db_session
    app.dependency_overrides[get_db] = override_get_db
    return TestClient(app)

```

```

# tests/test_api.py
import pytest
from datetime import datetime
from shortener.models import URL

class TestShortenURL:
    def test_shorten_returns_201_and_short_code(self, client):
        resp = client.post("/shorten", json={
            "url": "https://example.com/very/long/path"
        })
        assert resp.status_code == 201
        data = resp.json()
        assert "short_code" in data
        assert len(data["short_code"]) == 6

    def test_shorten_invalid_url_returns_422(self, client):
        resp = client.post("/shorten", json={
            "url": "not-a-url"
        })
        assert resp.status_code == 422

class TestRedirect:

```

```

def test_redirect_returns_301(self, client):
    resp = client.post("/shorten", json={"url": "https://example.com"})
    code = resp.json()["short_code"]
    resp = client.get(f"/{code}", follow_redirects=False)
    assert resp.status_code == 301
    assert resp.headers["location"] == "https://example.com"

def test_nonexistent_code_returns_404(self, client):
    resp = client.get("/nonexist")
    assert resp.status_code == 404

class TestStats:
    def test_stats_tracks_visits(self, client):
        resp = client.post("/shorten", json={"url": "https://example.com"})
        code = resp.json()["short_code"]
        client.get(f"/{code}")
        client.get(f"/{code}")
        resp = client.get(f"/stats/{code}")
        assert resp.status_code == 200
        data = resp.json()
        assert data["visits"] == 2
        assert data["original_url"] == "https://example.com"

```

测试策略：- 用内存 SQLite (`:memory:`) 而不是真正的数据库文件 - 每个测试独立的 session , 互不干扰 - 通过 FastAPI 的 `dependency_overrides` 机制替换真实数据库

```

pytest -v                                # 全部跑通
pytest --cov=shortener                    # 带覆盖率

```

九、运行

```

# 1. 安装 (开发模式)
pip install -e ".[dev]"

# 2. 初始化数据库
shortener init-db

```

```
# 3. 启动 API
uvicorn shortener.app:app --reload --port 8000
# http://localhost:8000/docs ← 自动 Swagger !

# 4. 测试
pytest -v --cov=shortener
```

十、扩展方向

到这里你已经有了一个可运行的短链服务。以下是真实世界需要但本课程没展开的：

功能	怎么做	涉及的新技术
自定义短码	支持用户指定 code	加一个可选的 <code>code</code> 字段
过期时间	短链到期自动 404	加 <code>expires_at</code> 字段 + 定时清理
API Key 认证	请求头校验	FastAPI Header + Depends
异步改造	async handler + async SQLAlchemy	async/await + asyncpg
Docker 部署	容器化	Dockerfile + docker-compose
前端页面	HTML 表单创建短链	Jinja2 模板或纯前端
数据库迁移	安全修改表结构	Alembic

🎉恭喜完成 7 天 Python 训练营！

你已经覆盖了：

Day	主题	核心工具
Day 1	基础语法速通	数据类型、函数、OOP、异常
Day 2	进阶语法	装饰器、生成器、上下文管理器
Day 3	标准库兵器谱	collections, itertools, logging, re
Day 4	OOP & 类型系统	property, ABC, Protocol, dataclass
Day 5	并发编程	threading, multiprocessing, asyncio
Day 6	工程化	pytest, FastAPI, SQLAlchemy, Click
Day 7	实战项目	URL Shortener 全栈

下一步建议：- 把这个项目真正部署到服务器上 - 读开源项目源码（FastAPI、uvicorn 都是精品） - 深入你需要的方向（后端、数据工程、DevOps） - 保持写 Pythonic 代码的习惯——能少写就少写

记住：Python 的优雅不是来自特性多，而是知道什么时候用什么。

```
import this
# The Zen of Python, by Tim Peters
# ...
# Simple is better than complex.
# ...
# Readability counts.
```